

A Framework for Temporal Consistency Checking of Sequential Chain-of-Causation Annotations

연속 인과관계(CoC) 주석의 시간적 일관성 검사를 위한 프레임워크

Abstract: Sequential Chain-of-Causation (CoC) annotations can appear consistent alone yet conflict across events when earlier requirements remain active or linger after ending. We propose a temporal consistency framework that extracts actions, continuing requirements, end conditions, action order, and true/false/unknown evidence from ordered CoCs. It tracks active requirements and records the first error and reason. Python performs the primary check; UPPAAL replays the same rules. Review of 309 adjacent pairs from 90 CoC-Nusc scenes, including 27 groups, showed ambiguity (Fleiss' kappa 0.027; 0/27 contradictions after discussion). In 40 reference-modified pairs, our checker detected all 40 modifications, versus 20 for a current-event-only checker; neither flagged a reference case. Seven UPPAAL cases matched Python decisions and records (0/7 mismatches). Under the tested rules, this supports temporal requirement tracking but not natural-error accuracy, vehicle safety, or learning gains.

Key Words: Autonomous driving, End-to-end learning, Chain-of-Causation, Temporal consistency checking, Model checking, Data-source tracking

1 서론

종단간(end-to-end, E2E) 자율주행은 센서 관측에서 미래 이동 경로나 제어 의도를 직접 예측하며, 최근 비전-언어 주행 모델은 행동 이유를 설명하는 Chain-of-Causation(CoC)을 함께 생성한다 [1, 2, 3, 4, 5]. CoC는 각 시점의 행동을 정당화하는 설명이지만, 생성·작성 과정에서 조건을 누락하거나 앞서 제시한 요구와 양립하지 않는 행동을 설명할 수 있다. 같은 주행 구간의 CoC들이 개별적으로는 자연스러워도 시간순으로 함께 읽으면 모순이 드러날 수 있는 것이다. 이러한 불일치 때문에 학습 자료의 행동 요구가 모호해지고, 같은 자료를 다시 검사했을 때 일관된 결과를 얻기 어렵다.

다음은 연구 동기를 설명하기 위한 가상 예시이며 자연 자료에서 발견한 오류 사례는 아니다. 앞선 CoC가 “보행자가 통과할 때까지 정지한다”고 하고, 후속 CoC가 “신호가 녹색으로 바뀌었으므로 출발한다”고 했다고 하자. 녹색 신호는 출발의 새로운 근거이지만 보행자가 통과했다는 증거는 아니다. 보행자가 여전히 횡단 중임이 확인되면 출발은 이전 정지 요구와 충돌한다. 반면 통과 여부가 제시되지 않았다면 확정 모순이 아니라 정지 요구의 해제 근거가 부족한 경우이다. 보행자의 통과가 확인된 뒤 출발하면 해당 요구에 관한 충돌은 없다. 따라서 새로운 행동 근거가 있다는 사실만으로 이전 요구를 지워서는 안 된다.

본 연구의 동기는 이러한 시간 순서와 조건에 따른 모순을 UPPAAL 모델로 검증하고, 문제를 일으킨 사건과 근거를 추적하는 것이다. 이를 위해 연속 CoC에서 계속 지켜야 할 요구, 그 요구가 끝나는 종료 조건과 행동을 구조화하고, 이후 사건의 증거가 어느 요구에 연결되는지 구분한다. 이전 요구를 기억하는 검사는 현재 행동을 남아 있는 요구와 비교하며, 이해제가 확인된 경우와 종료 여부를 알 수 없는 경우를 구별한다. 이때 검증하는 것은 명시된 문장과 증거 아래의 일관성이다. CoC에 처음부터 언급되지 않은 위험까지 빠짐없이 설명했는지 또는 실제 주행이 안전했는지를 판정하는 것은 아니다.

그림 1은 자료 정리, 구조화, 상태 갱신과 결과 추적의 흐름을 보여 준다. UPPAAL 모델은 주어진 사건 순서와 요구의 유지·종료 조건을 따라 위반 상태에 도달하는지 검사한다. Python 검사기는 같은 규칙의 실행과 결과 집계를 담당하며, 두 구현의 최종 판정, 오류 종류, 최초 위반 사건과 정보 부족 사유를 비교한다. 이러한 비교는 고정된 규칙의 구현 일치성을 확인하는 절차이며, 자연어 해석이나 규칙 자체의 타당성을 독립적으로 증명하지 않는다. 또한 원문의 조건을 찾는 과정과 그 조건의 참·거짓 증거를 확보하는 과정은 구분해야 한다. 현재

자동 변환기는 행동과 조건 문구를 추출하지만 관측된 조건의 충족·해제 여부를 자동으로 확정하지 않으므로, 구조화 입력의 검사 결과를 곧바로 원문 자동 탐지 성능으로 해석하지 않는다.

본 논문의 연구 질문은 다음과 같다.

연구 질문: 연속 CoC의 요구와 종료 근거를 연결한 모델에서 행동 전환의 모순과 근거 부족을 구분하고, 최초 문제 사건과 그 이유를 추적할 수 있는가? 이전 요구를 기억하는 검사는 현재 사건만 보는 검사와 어떻게 다르며, 같은 입력에 대한 Python과 UPPAAL의 판정 및 위반 기록은 일치하는가?

이 질문에 답하기 위해 자연 주석 검토, 통제된 합성 규칙 시험과 구현 일치성 확인을 구분한다. 자연 주석 검토는 사람이 실제 설명을 얼마나 일관되게 해석하는지 살핀다. 통제 시나리오는 요구의 이해, 정상 해제와 근거 누락을 구분하여 검사 가능성과 오류 추적을 확인한다. 현재 중심 구현은 한 개의 활성 요구를 관리하며, 복수 요구의 중첩은 지원 단계로 다룬다. 종료된 요구를 잘못 남기는 검사 처리 문제도 입력 CoC의 모순과 구분하여 해석한다.

중심인 연속 문장 검사와 별도로, 보조인 반응 시간 검사는 개별 CoC가 요구한 행동과 기록된 자차 속도 변화가 정해 둔 시간 안에 맞아떨어지는지 확인한다. 문장 사이의 일관성과 실제 속도 반응은 다른 질문이므로 하나의 정확도나 안전 점수로 합치지 않는다. 자료상 연속성이 확인된 같은 주행 구간 안에서만 요구를 이어받으며, 연결 근거가 없는 서로 다른 장면에는 요구를 넘기지 않는다.

핵심 기여는 세 가지이다. 첫째, 연속 CoC의 행동 요구와 종료 조건을 구분하고 후속 증거를 앞선 요구에 연결하는 검사 표현을 제시한다. 둘째, 이 표현을 Python 검사기와 UPPAAL 모델로 실행하여 판정, 오류 종류, 최초 위반 사건과 정보 부족 사유를 기록한다. 셋째, 자연 자료의 해석 문제와 통제 시나리오의 규칙 실행 결과를 분리하여 프레임워크의 적용 가능성과 한계를 보고한다. 본 논문은 자연 오류율, 물리적 차량 안전 또는 학습 성능 향상을 주장하지 않는다.

2 관련 연구

DriveLM, DriveVLM과 Alpamayo-R1은 장면 이해와 행동·궤적 예측을 언어 추론에 연결한다 [3, 4, 5]. 최근 연구는 생성 설명이 장면과 행동에 충실한지, 센서 교란이나 객체 제거에도 추론-행동 관계가 유지되는지를 평가한다 [6, 7, 8, 9].

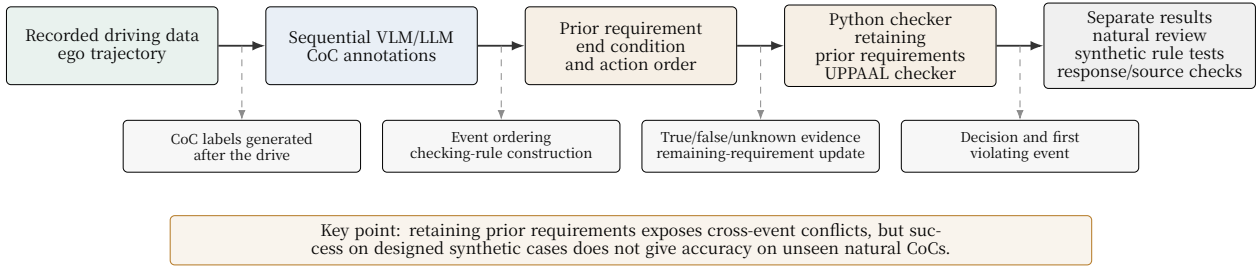


그림 1. 연속 CoC에서 지속 요구, 종료 조건, 행동 순서와 미확정 증거를 추출하여 이전 요구를 기억하는 검사 규칙으로 변환하는 전체 흐름. 자연 표본 검토와 합성 규칙 시험은 서로 다른 증거로 보고한다.
Fig. 1. Flow for turning sequential CoCs into rules that retain prior requirements, with natural review separated from controlled synthetic tests.

이들의 주된 단위는 한 장면의 추론·행동 정렬 또는 입력 개입에 대한 모델 반응이다.

본 연구는 모델에 같은 질문을 다시 하거나 차량 행동을 교정하지 않는다. 이미 저장된 CoC를 시간순으로 읽어 이전 사건에서 시작된 요구, 그 요구가 끝나는 조건과 행동 순서를 검사한다. 따라서 한 장면의 설명이 맞는지 평가하는 연구를 대체하지 않고, 여러 주석을 이어 읽었을 때 서로 모순되지 않는지 확인하는 보완적 품질 점검에 해당한다.

입력이나 가정을 일부 바꾸어 결과 변화를 보는 반사적 추론 연구는 행동 분류를 수정하거나 구조화된 설명 레이블을 학습에 넣어 계획을 교정한다. 위반 사례를 새로 만들어 모델을 개선하는 연구도 있다. 이와 달리 본 연구는 새 행동이나 학습 레이블을 생성하지 않으며, 검사 규칙을 통과하지 못한 항목을 곧바로 위험 자료로 바꾸지 않는다. 출력은 어떤 요구·처리 방식·증거 때문에 해당 판정이 나왔는지 함께 기록한 검토 후보이다.

UPPAAL은 시간에 따라 상태가 바뀌는 규칙을 자동으로 검사하는 도구이며, Scenic과 VerifAI는 자율 시스템의 시험 상황 생성과 위반 탐색을 지원한다 [10, 11, 12]. 본 논문은 새로운 환경이나 제어 방법을 찾는 데 UPPAAL을 쓰지 않고, 기록된 사건을 순서대로 재생하며 요구의 생성·유지·종료를 확인하는 데 사용한다. 따라서 기여는 차량과 환경의 반복 상호작용 전체에 대한 안전 보증이 아니라, 저장된 연속 주석을 자동 실행하고 위반 이유를 추적할 수 있는 검사 규칙으로 만든 데 있다.

정리하면 기존 연구는 주로 한 장면의 설명이 행동과 맞는지, 입력을 바꾸면 행동도 달라지는지, 또는 실제 실행 시스템이 안전 조건을 지키는지를 묻는다. 본 연구는 같은 장면에 저장된 여러 CoC를 이어 읽으며 앞선 요구가 언제 시작되고 계속되며 끝나는지를 묻는다. 관련 연구는 이 질문의 차이를 설명하는 범위로 한정하고, 본 실험에서 직접 비교하지 않은 방법 간 우열은 논하지 않는다.

표 1은 어떤 방법이 더 우수한지를 비교하기보다 각 연구가 무엇을 근거로 어떤 질문에 답하는지를 구분한다. 장면별 설명 연구가 “설명이 장면과 행동에 맞는가”를 묻는다면, 본 연구는 “한 사건의 요구가 다음 사건까지 남을 때 두 주석이 함께 성립할 수 있는가”를 묻는다. 또한 실제 환경과 차량 제어를 함께 실행하여 시험하는 연구와 달리, 본 연구는 저장된 문장과 기록만 검사한다. 따라서 제어 결과가 다시 차량과 환경에 영향을 주는 전체 과정의 안전을 보장하지 않는다.

3 데이터와 증거 범위

본 연구는 nuScenes 주행 기록을 실험용 형식으로 바꾸고 CoC 자동 생성 결과를 결합한 CoC-Nusc 자료를 사용한다 [13, 14]. 실제 입력은 변환된 카메라 영상 묶음, 장면 안의 자차 움직임과 주행이 끝난 뒤 생성한 CoC이다. 자동 생성기는 먼저 기록 궤적에서 감속·정지·진행 같은 행동 분류를 계산하고, 그 정보를 시각·언어 모델(VLM/LLM)에 제공해 CoC를 만든다. 따라서

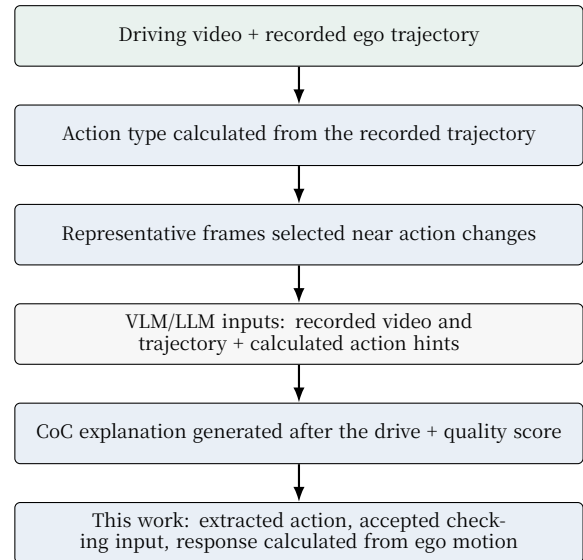


그림 2. 기록 영상과 미래 궤적에서 자동 CoC 주석을 만들고 검사 입력으로 바꾸는 순서. 행동 종류를 기록 궤적에서 먼저 계산하므로 CoC와 궤적이 일치해도 올바른 원인을 찾았다고 단정할 수 없다.
Fig. 2. How automatic CoC annotations are generated after a drive and converted into checking inputs.

이 CoC는 사람이 직접 확정된 정답이나 주행 중 제어기의 판단이 아니라, 기록을 사후에 설명한 자동 생성 주석이다.

표 2은 각 자료가 어디에서 왔고 어떤 결론까지 허용하는지를 보여 준다. CoC 문장과 품질 점수는 센서가 직접 측정된 값이 아니라 기계가 만든 주석이며, 행동 종류와 검사 결과도 본 연구가 정한 규칙으로 계산한 값이다. 따라서 검사 통과는 정한 일관성 오류를 찾지 못했다는 뜻이지, 원천 기록이 모두 사실이거나 차량이 안전했다는 인 증은 아니다.

그림 2에서 자동 생성기는 기록 궤적에서 행동 종류와 행동이 바뀌는 대표 시점을 먼저 고른 뒤, 영상·궤적·행동 정보를 이용해 CoC를 만든다. 즉 CoC는 주행 중 제어기가 내린 판단이 아니라 이후의 움직임까지 본 뒤 작성된 설명이다. 그래서 CoC와 기록 궤적이 맞다는 사실만으로 모델이 올바른 원인을 찾았다고 보지 않으며, 문장 사이의 모순과 반응 시간을 검사하는 입력으로만 사용한다.

자차 궤적과 연결되는 자료는 94장면·403개 CoC 사건이다. 이 중 CoC가 두 개 이상인 90장면에서 시간상 이웃한 문장 쌍 309개를 만들었다. 사람 검토에는 자동으로 고른 후보 15장면과 이들과 겹치지 않는 비교 대상 12장면, 총 27개 장면 묶음을 사용했다. 따라서 309는 자동 추출한 전체 문장 쌍 수이고, 27은 무작위로 뽑지 않은 사람 검토 대상 수이다.

한 사건의 요구와 자차 반응 시간을 비교하는 보조 분석은 대상 수가 다르다. 403개 사건 중 기계 품질 기준을 통과한 290개를 먼저 남기고, 그중 감속·가속 방향이 분명한 134개만 검사했다. 방향이 모호하거나 여러 행동이 섞인 156개는 제외했다. 134건

표 1. 핵심 관련 연구군과 본 연구의 주된 평가 단위 및 주장 범위.
Table 1. Primary evaluation units and claim scopes of the closest research families and this work.

Research area	Main item studied	Typical test or input	Relation to this work
CoC explanation quality and action response [6, 7, 8, 9]	Match between reasoning and action in one scene	Run the model again with changed input or a removed object	Shows whether an explanation matches the scene and action
Reasoning-based model improvement	Alternative action or reasoning label	Change planning or training	Changes model behavior; this work only checks stored annotations
Automatic autonomy-system testing [10, 11, 12]	A driving situation or a stated system rule	Search environments, simulate, or check a rule	Provides checking tools; this work replays fixed annotation events
This work	Requirements carried across stored CoCs	Build rules, retain prior requirements, and compare fixed cases	Checks annotation consistency and data sources, not physical safety

표 2. 평가 자료의 출처와 본 논문에서 허용되는 해석.
Table 2. Data sources and permitted interpretation of the evaluation data.

Source	Content	Role	What it can show
nuScenes	Sensor records and ego-vehicle state	Original driving record	What the sensors recorded
CoC-Nusc processing	Aligned camera images and ego-motion table	Match records within a scene	Values calculated from the records
Automatic CoC generator	Explanation text and quality score	Machine-generated sequential annotation	Explanation generated after the drive
This work	Shared checking format, checker, and test cases	Check consistency across events	Decision under the stated rules

표 3. 증거 수준과 허용되는 주장.
Table 3. Evidence levels and permitted claims.

Source type	Example	What it can show
Recorded	CoC text, time, ego-motion row	The record exists
Calculated	Extracted action or check result	A result under the stated rules
Researcher-defined rule	Requirement-ending and memory rules	How the proposed checker behaves
Controlled modified case	One deliberately changed item	A known rule is exercised

가운데 130건은 이후의 속도 변화를 찾아야 했고, 4건은 사건 시점에 이미 정지·양보 조건을 만족했다. 반복된 CoC가 같은 반응을 가리킬 수 있으므로 134를 서로 완전히 독립된 주행 사례 수로 해석하지 않는다. 이 선별 과정은 중심 분석의 403개 사건과 309개 문장 쌍에는 적용하지 않았다.

검토자는 시간 조건이 분명한지, 요구된 행동 순서가 무엇인지, 문장들이 서로 모순되지 않는지를 판정했다. 보행자 통과처럼 자료에 없는 외부 상태가 필요하면 추측하지 않고 미상으로 남겼다. 기록 궤적은 문장과 움직임을 따로 비교하는 데만 사용했으며, 사람 운전 기록을 곧바로 안전 정답으로 보지 않았다. 센서 기록, 그 기록에서 계산한 값, 연구자가 정한 검사 규칙과 의도적으로 만든 변형 사례는 서로 구분해 보고한다.

검사 통과는 기록의 사실성이나 안전성을 새로 인증하지 않는다. 자료에 없는 요구 종료 여부와 외부 객체 상태는 미상으로 유지하며, 의도적으로 만든 변형 사례의 판정을 자연 자료의 오류 정답으로 사용하지 않는다.

4 문제 정의

본 논문은 서로 다른 두 질문을 따로 검사한다. 중심인 연속 문장 검사는 이전 CoC의 요구와 종료 조건을 기억하여 앞뒤 문장과 검사 상태가 정한 규칙에 맞는지 확인한다. 보조인 반응 시간 검사는 문장에서 읽어 낸 행동 요구와 기록된 자차 속도 변화가 정해 둔 시간 안에 맞는지 확인한다. 첫 번째는 문장과 요구 상태의 일관성, 두 번째는 요구 시점과 속도 반응 시점의 관계를 다루므로, 한 검사를 통과해도 다른 검사까지 통과했다는 뜻은 아니다.

수식에 앞서 기호의 뜻을 설명한다. 장면 s 의 사건을 시간순으로 늘어놓은 목록이 $E_s = (e_1, \dots, e_n)$ 이다. 사건 e_i 직전과 직후에 아직 지켜야 하는 요구의 집합을 각각 A_{i-1} 과 A_i 라 한다. 행동 표현은 정지·유지, 양보·감속, 가속·진행, 현재 행동 유지의 네 종류로 묶는다. 요구 o 가 끝났다는 증거 $q_i(o)$ 는 참, 거짓,

미상 중 하나이며, 미상일 때는 요구를 임의로 지우지 않는다. $\text{sat}(e_i)$ 는 현재 사건에 적힌 선행조건이 충족됐는지를 뜻하고, A_i^- 는 종료 증거를 먼저 반영한 뒤 현재 행동을 실행하기 직전의 요구 집합이다.

다섯 검사 문제는 다음과 같다. P1은 이전 요구와 현재 행동의 충돌, P2와 P3은 한 사건 안의 종료·순서 규칙 위반, P4는 끝난 요구를 목록에 남긴 검사 처리 오류이다. 본 논문은 P1-P4를 묶어 일관성 오류라고 부른다. P5는 이들과 별개로 요구와 자차 반응의 시간 차이를 검사한다.

P1: 끝나지 않은 요구와 진행의 충돌. $C_1(i) \equiv \text{go}(e_i) \wedge \exists o \in A_i^- : \text{hold}(o)$. 아직 끝나지 않은 정지·양보 요구가 있는데 진행을 시작했는지 판정한다.

P2: 조건 충족 전 종료. $C_2(i) \equiv \exists o : \text{release}(e_i, o) \wedge q_i(o) = \text{False}$. 종료 조건을 아직 만족하지 않았는데 요구를 끝냈는지 판정한다.

P3: 행동 순서 위반. $C_3(i) \equiv \text{go}(e_i) \wedge \text{sat}(e_i) = \text{False}$. 현재 사건이 필수 선행조건의 미충족을 명시하면서 후속 행동을 시작했는지 판정한다.

P4: 종료 뒤 남은 요구. $C_4(i) \equiv \exists o \in A_{i-1} : q_i(o) = \text{True} \wedge o \in A_i$. 직전까지 남아 있던 요구가 종료 조건을 만족한 뒤에도 사라지지 않았는지 판정한다.

P5: 제한 시간 내 반응. 검사 대상으로 받아들인 사건의 시각 τ_i , 요구 행동 a_i , 시험용 제한 시간 D_i 와 그 행동에 맞는 속도 변화 조건 R_{a_i} 에 대하여

$$\exists t_r \in [\tau_i, \tau_i + D_i) : R_{a_i}(t_r) \quad (1)$$

의 만족 여부를 판정한다.

P1-P4 중 가장 먼저 발생한 위반 위치는

$$i^* = \min\{i \mid \bigvee_{k=1}^4 C_k(i)\} \quad (2)$$

로 정한다. P1-P4가 한 번도 발생하지 않으면 문장열 E_s 를 일관된 것으로 판정한다. 하나라도 발생하면 가장 먼저 발생한 위치 i^* 와 오류 종류를 함께 반환한다. 현재 사건만 보는 검사는 매 사건에서 이전 요구를 비우고($A_{i-1} = \emptyset$), 이전 요구를 기억하는 검사는 A_{i-1} 을 다음 사건으로 전달한다.

표 4. 현재 사건만 보는 검사와 이전 요구를 기억하는 검사의 차이.
Table 4. Difference between checking only the current event and retaining prior requirements.

Item	Current event only	Prior requirements retained
Input	Current event	Ordered event sequence
Prior requirement	Discarded	Retained until its end condition
Missing evidence	Recorded only for the event	Requirement and reason are retained
Output	Error found in that event	Decision, error type, and first violating event

위 표의 핵심 차이는 이전 요구를 다음 사건으로 넘기는지 여부이다. P5의 반응 시간 t_r 은 자차 속도를 W 길이의 구간으로 살펴보고, 속도 변화가 최소값 δ 를 넘으며, 그 변화 방향이 요구 행동과 ρ 비율 이상 맞을 때 정한다. 기본값은 $W = 0.5$ 초, $\delta = 0.1$ m/s, $\rho = 0.7$, $D = 3$ 초이며 법규나 안전 기준에서 정한 값은 아니다. 반복 CoC에는 첫 판단 시각 유지, 반복마다 시계 재시작, 중복 사건 제거의 세 방식을 비교한다. 두 장면이 실제로 이어진다는 자료가 없으면 이전 장면의 반응 요구를 다음 장면으로 넘기지 않는다.

따라서 본 문제는 미리 정한 규칙 아래에서 CoC 문장과 검사 상태가 앞뒤로 일관되는지, 그리고 요구 행동과 기록된 반응 시점이 맞는지를 확인한다. 장면 설명이 실제로 참인지, 기록 궤적이 안전한지, 법규를 지켰는지 또는 제어 성능이 좋은지는 판정하지 않는다.

5 제안 방법

제안 절차는 네 단계로 구성된다. 첫째, 자료 정리 단계에서 같은 장면 식별 번호(자료 파일의 clip_id)를 가진 CoC를 시간순으로 놓고 각 값의 출처를 보존한다. 둘째, 규칙 변환 단계에서 문장 속 행동, 계속 지켜야 할 요구, 그 요구가 끝나는 조건과 행동 순서를 공통 형식으로 만든다. 셋째, 이전 요구를 기억하는 검사기가 사건마다 남은 요구를 갱신하고 오류 종류와 첫 위반 사건 번호를 기록한다. 넷째, 자연 자료, 합성 규칙 시험과 Python-UPPAAL 일치 결과를 서로 나누어 보고한다.

자료 정리 단계에서는 문장이 맞는지 또는 차량이 위험했는지를 판단하지 않는다. 센서에서 직접 얻은 입력, 기계가 매긴 품질 점수와 자차 속도에서 계산한 반응을 구분해 저장하고, 장면과 사건 순서를 고정한다. 규칙 변환 단계에서는 “정지한다”, “통과할 때까지”, “그 뒤 진행한다”처럼 여러 단계가 있는 문장을 마지막 행동 하나로 줄이지 않는다. 대신 요구 행동, 계속되는 요구, 종료 조건과 선행 행동으로 나누고, 각 항목이 원문의 어디에서 왔는지와 문장을 해석하지 못한 이유를 남긴다.

검사기는 새 요구가 나오면 아직 지켜야 할 목록에 넣고, 요구가 끝났다는 명시적 증거가 참이면 목록에서 지운다. 종료 조건이 거짓이면 요구를 유지하고, 판단할 증거가 없으면 미상과 그 이유를 함께 남긴다. 마지막 단계에서는 자연 자료 판정, 합성 시험 사례 판정, Python-UPPAAL 일치, 보조 반응과 자료 출처 결과를 별도로 보고한다. 이렇게 하면 대상 수와 질문이 다른 결과가 하나의 성능 수치로 잘못 합쳐지는 것을 막을 수 있다.

그림 3의 위쪽은 공통 입력을 검사 규칙과 판정 기록으로 바꾸는 과정이고, 아래쪽은 서로 다른 평가 자료를 뜻한다. 자연 자료에 대한 사람 판정은 합성 변형을 만들거나 검사 기준을 조정하는 데 사용하지 않는다. 반대로 합성 사례의 기대 판정도 자연 자료의 정답으로 사용하지 않는다. 따라서 40쌍이 정해 둔 규칙을 모두 실행했다는 결과와 자연 오류를 얼마나 잘 찾는지 구분된다.

사건 e_i 를 읽기 전에 아직 지켜야 하는 요구의 집합을 A_{i-1} 이라

표 5. 이전 요구를 기억하는 검사가 확인하는 네 일관성 오류.
Table 5. Four consistency-error types checked while retaining prior requirements.

Consistency error	Condition that causes it
Hold-go conflict	Proceed while a stop or yield requirement remains
Requirement ended too early	End a requirement before its end condition is met
Action order violation	Start the next action before the required earlier action
Requirement left after its end	Keep a requirement after its end condition is met

표 6. 평가에 사용한 두 UPPAAL 검사 모델과 각 모델의 확인 대상.
Table 6. Two UPPAAL checking models and what each checks.

Model	Checking components	What is checked
Sequential CoC test cases	Requirement-memory checker	Order, error type/location, missing information
scene-0001 response model	Event replay and three checks	Start/end, deadline, repeat clock, overwrite

하면, 상태를 갱신하는 함수 T 는 다음 요구 집합과 판정을 계산한다.

$$(A_i, V_i) = T(A_{i-1}, e_i, q_i), \quad (3)$$

여기서 $q_i \in \{\text{True}, \text{False}, \text{Unknown}\}$ 는 요구 종료나 외부 객체 상태에 대한 참·거짓·미상 증거이고, V_i 는 정상 판정 또는 일관성 오류의 종류이다. 현재 사건만 보는 검사는 매번 이전 요구를 비우지만($A_{i-1} = \emptyset$), 이전 요구를 기억하는 검사는 이를 다음 사건으로 넘긴다. 이 차이 때문에 후자는 정지 요구가 끝나기 전에 진행하는 경우와, 끝난 정지 요구를 검사 목록에서 지우지 않은 경우를 판정할 수 있다.

같은 사건 목록은 UPPAAL 검사 모델로도 변환된다. 이 모델은 사건을 하나씩 읽으면서 요구가 시작되고, 이어지고, 끝나는 과정과 위반 여부를 갱신한다. 한 사건의 상태 갱신이 끝날 때까지 다음 사건을 처리하지 않는 UPPAAL 기능을 사용하여 사건 순서를 고정한다. 각 단계가 오류 없이 이어지는지와 오류 상태에 도달하는지를 확인하고, 처음 위반한 사건 번호를 실행 기록에 남긴다. 이 모델은 기록된 문장 순서만 검사하며 실제 차량 움직임이나 환경을 새로 만들어 시험하지 않는다.

Python 검사기와 UPPAAL 검사 모델은 같은 사건과 증거를 입력받는다. Python은 장면별 판정과 남아 있는 요구의 변화를 기록하고, 변환 프로그램은 같은 내용을 UPPAAL 모델 파일에 넣는다. 두 결과를 비교할 때는 최종 정상/오류 판정, 오류 종류, 첫 위반 사건과 미상 사유를 확인한다. 예를 들어 정지 요구가 끝나기 전에 진행한 사례에서는 정지 요구가 시작되어 목록에 남고, 이후 진행 사건을 읽을 때 오류로 바뀌는 순서를 확인할 수 있다. 따라서 정상/오류만 표시할 때보다 어떤 사건에서 왜 오류가 났는지 추적하기 쉽다.

두 UPPAAL 모델의 역할은 다르다. 중심 검사 모델은 연속 CoC의 순서와 남아 있는 요구를 확인하고, 그림 4의 보조 모델은 scene-0001에서 반응 시간, 반복 사건과 자료 출처 처리를 확인한다. 표 6처럼 두 모델을 나누어 보고하여, 반응 시간 시험이 중심 문장 검사를 다시 검증한 결과로 오해되지 않게 한다.

이제 보조 반응 검사를 여러 역할로 나눈 UPPAAL 모델을 설명한다.

그림 4에서 CoC 사건 재생 부분은 최초 판단, 반복 판단과 품질 기준을 통과하지 못한 사건을 시간순으로 읽고, 자차 움직임 재생 부분은 속도에서 찾은 반응을 읽는다. 세 검사 부분은 각각 요구의 시작·유지·종료, 반응 제한 시간과 반복 시계, 낮은 품질 사건이

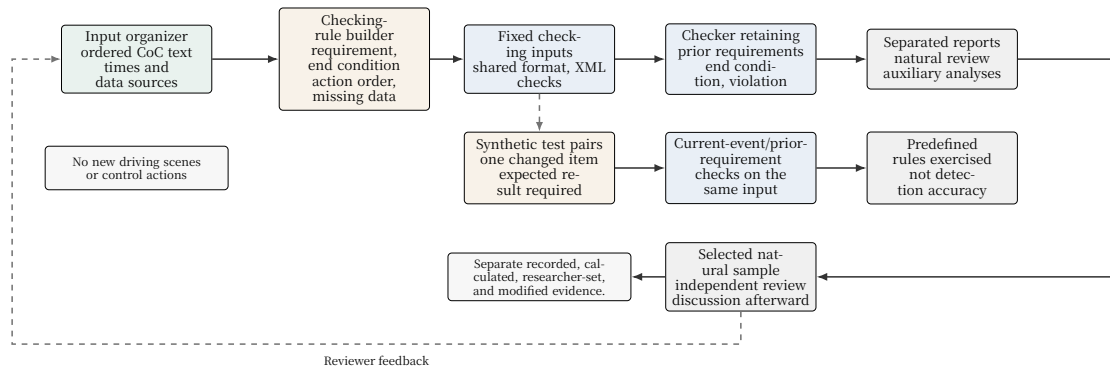


그림 3. 자료 정리, 검사 규칙 변환, 이전 요구를 기억하는 판정과 결과 분리 절차. 자연 자료 검토와 기대 판정을 미리 확인한 합성 규칙 시험은 별도 경로로 유지한다.
Fig. 3. Data organization, rule construction, history-aware checking, and separate paths for natural review and controlled synthetic tests.

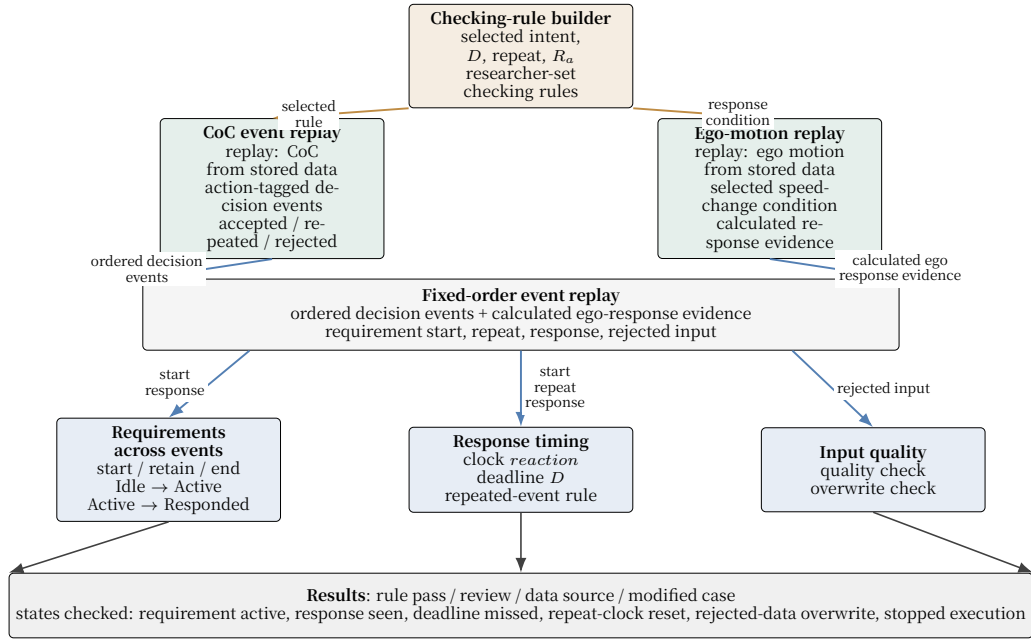


그림 4. scene-0001의 기록 사건 재생과 요구·반응·자료 출처 검사를 역할별로 나눈 UPPAAL 모델.
Fig. 4. UPPAAL model separating fixed event replay from requirement, response, and data-source checks for scene-0001.

표 7. scene-0001의 기록값·계산값과 UPPAAL 사건의 대응 관계.
Table 7. Mapping recorded and calculated scene-0001 evidence to UPPAAL events.

Recorded or calculated evidence	Event given to the model	Meaning
Accepted deceleration at 2.500 s	First requirement at 0.000 s	Start the requirement
Repeated deceleration at 3.500 s	Repeated requirement at 1.000 s	Record a repeated event
Speed-decrease onset at 4.660 s	Response at 2.160 s	End with the calculated response
Repeated deceleration at 4.800 s	Repeated requirement at 2.300 s	Record a repeat after response
Low-quality CoC at 9.300 s	Rejected input at 6.800 s	Do not overwrite accepted data

기존 정보를 덮어쓰는지를 확인한다. 이 모델은 새 행동이나 환경을 만들지 않고, 기록된 사건을 읽었을 때 검사 상태가 의도한 순서로 바뀌는지만 확인한다.

표 7은 원래 기록된 시각과 검사 모델이 읽는 사건의 관계를 보여 준다. 첫 감속 CoC에서 지켜야 할 요구가 시작되고, 두 번의 반복 CoC는 같은 요구에 연결된다. 자차 속도에서 계산한 반응이 확인되면 그 요구를 끝낸다. 마지막의 낮은 품질 사건은 앞서 받아들인 정보를 덮어쓰지 않아야 한다. 센서가 기록한 값, 기록에서 계산한 값, 연구자가 정한 값을 표에서 구분하여 각 값의 출처를 확인할 수 있게 했다.

자연 자료와 별도로 네 일관성 오류마다 정상 기준 사례 10개를

만들고, 각 기준 사례에서 한 항목만 바꾼 사례를 짝지었다. 생성 프로그램은 기준 사례에는 오류가 없고 바꾼 사례에는 미리 지정한 오류 하나만 나타나는지 확인한 뒤에만 그 쌍을 사용했다. 따라서 합성 40쌍은 정해진 네 규칙이 의도대로 작동하는지 확인하는 시험 사례이다. 기대 판정이 나오도록 생성 단계에서 미리 결렸으므로, 처음 보는 자연 오류를 얼마나 잘 찾는지 평가하는 자료는 아니다.

한 항목만 바꾸면 어떤 변화가 어떤 오류를 만들었는지 분명해진다. 첫째, 정지·양보 요구가 끝났다는 증거 없이 진행을 시작한다. 둘째, 종료 조건이 아직 거짓인데 요구를 끝낸다. 셋째, 꼭 먼저 해야 하는 행동보다 다음 행동을 앞세운다. 넷째, 종료 조건이 참인데도 끝난 요구를 목록에서 지우지 않는다. 두 검사에는 같은 현재 사건 정보를 주고, 현재 사건만 보는 검사에만 이전 요구를 전달하지 않았다. 따라서 판정 차이는 문장 해석이나 입력 차이가 아니라 이전 요구를 기억하는지에서 생긴다.

Python과 UPPAAL 비교에는 정상 사례 하나, 네 오류 사례, 증거 부족 사례와 문장 해석 실패 사례 등 일곱 표준 시험 사례를 사용한다. 최종 판정, 오류 종류, 첫 위반 사건과 미상 사유를 비교한다. 두 구현이 같은 판정 규칙을 사용하므로, 이 비교는 규칙 자체가 옳다는 독립 증명보다 두 구현이 같은 결과와 기록을 내는지 확인하는 시험이다.

표 8. Python-UPPAAL 구현 일치 확인에 사용한 표준 시험 사례.
Table 8. Standard test cases for Python-UPPAAL implementation agreement.

Test-case group	Count	Compared output
Consistent sequence	1	Consistent decision
Four consistency errors	4	Error type and first violating event
Missing evidence/text interpretation	2	Insufficient-information decision and reason
Total	7	Final decision and decision-record items

표 9. 평가 블록별 입력, 측정과 해석 경계.
Table 9. Inputs, measurements, and interpretation boundaries of the evaluation blocks.

Evaluation	Evidence and comparison	Interpretation boundary
Main consistency	309 windows/90 scenes; 27 reviews; 40 pairs; 7 test cases	Cross-event consistency; no natural-error rate, independent accuracy, or safety
Fixed-scene check	scene-0001; normal vs. four modified cases	Known-error check in one scene; not natural-error performance
Response settings	94 scenes/403 CoCs; repeat rule, $D = 1-5$ s, 27 settings	Result changes under checking settings; not safety classification
Scene boundary	Two-scene boundary; discard vs. carryover	Deliberate boundary-error check; not long-horizon evidence

표 10. 연속 CoC 자연 검토의 표본 구성과 판정 절차.
Table 10. Sampling and review protocol for the natural sequential-CoC study.

Item	Count
Multi-CoC scenes	90
Within-scene adjacent windows	309
Automatically selected candidate scenes	15
Non-overlapping comparison scenes	12
Scene groups reviewed independently	27
Independent reviewers	4

6 실험 설계

표 9는 네 평가가 무엇을 입력으로 쓰고 무엇을 확인하는지 정리한다. 중심 검증은 이전 요구를 기억하는 효과와 Python-UPPAAL 결과 일치를 확인한다. 나머지 세 평가는 한 장면에 의도적으로 넣은 오류, 반응 시간 설정에 따른 판정 변화, 장면 경계 처리를 보조적으로 확인한다. 분석 대상 수와 질문이 다른 결과는 하나로 합산하지 않는다.

중심 검증 안에서도 세 증거를 구분한다. 자연 자료는 같은 주석을 여러 사람이 얼마나 비슷하게 해석하는지 보여 준다. 합성 쌍은 같은 입력에서 이전 요구를 기억하는지에 따라 판정이 어떻게 달라지는지 보여 준다. 일곱 표준 시험 사례는 같은 판정 규칙을 구현한 Python과 UPPAAL이 최종 판정과 위반 기록을 똑같이 만드는지 확인한다.

표 10에서 사람 검토의 단위는 문장 쌍 하나가 아니라 장면 묶음이다. 자동 후보에서는 장면마다 하나만 택하고, 후보 장면과 겹치지 않는 비교 장면을 함께 골랐다. 따라서 같은 장면의 여러 문장 쌍을 서로 독립된 사례처럼 중복 집계하지 않는다. 다만 무작위 표본이 아니므로 27장면의 결과로 90장면이나 전체 자료의 모순 발생률을 추정하지 않는다.

네 검토자는 어떤 장면이 자동 후보인지, 비교 대상인지, 검사기가 어떤 결과를 냈는지 모른 채 27장면을 각각 판정했다. 시간 조건, 요구된 행동 순서와 문장 사이의 모순 여부를 먼저 독립적으로 평가한 뒤, 의견이 다른 항목은 근거를 함께 검토해

표 11. 중심 결과와 직접 지지되는 주장.
Table 11. Primary results and directly supported claims.

Result	Supported claim	Not supported
40 synthetic pairs: prior-requirement checker found 40/40 modified cases; current-only found 20/40; both found 0/40 reference cases Python-UPPAAL mismatch 0/7	Retaining prior requirements finds both cross-event error types; all four test rules were exercised	Accuracy on unseen natural errors or vehicle safety
27 selected scenes: textual consistency disputed in 23/27; $\kappa = 0.027$; contradiction 0/27 after discussion	Both implementations give the same decisions and reasons on tested cases Ambiguity shows the need for reviewer discussion; 0/27 is the selected-group record	Independent proof of the rules or vehicle safety Natural contradiction rate or detection accuracy

합의했다. 이 합의는 선택한 장면에 대한 검토자들의 공동 기록이며, 별도로 만든 자연 오류 정답 자료는 아니다.

합성 시험은 정지·양보 요구가 끝나기 전 진행, 조건 충족 전 요구 종료, 행동 순서 위반, 끝난 요구를 목록에 남긴 오류에 각각 10쌍을 사용한다. 두 검사에는 같은 입력을 주고, 각 오류를 몇 건 판정했는지와 정상 기준 사례를 오류로 잘못 판정했는지를 센다. 이 사례들은 무작위 표본이 아니므로 실제 자료의 오류를 얼마나 정확히 찾거나 놓치지 않는지는 계산하지 않는다.

기록된 전후 방향 움직임과 검토자 합의 결과의 관계는 별도로 살펴본다. 상대 차량·보행자 상태와 가속·제동 장치 명령이 없으므로, 문장의 요구와 자차 움직임이 맞지 않는 경우를 곧바로 안전 위반으로 해석하지 않는다.

중심 검증과 별도로 세 보조 평가는 프로그램과 설정이 의도대로 작동하는지 점검한다. 고정 장면 점검에서는 scene-0001의 최초 감속, 두 반복 판단, 자차 속도 반응과 낮은 품질 사건을 그대로 사용한다. 변경하지 않은 기준 모델과 제한 시간 축소, 반복 시 시간 다시 재기, 반응 제거, 낮은 품질 사건의 정보 덮어쓰기 사례를 비교한다. 각 사례는 관련 검사 조건을 하나 이상 실패해야 하므로, 한 장면에 의도적으로 넣은 네 오류를 검사기가 구분하는지 알 수 있다.

반응 설정 분석은 고정 장면 점검을 전체 자료로 넓힌 것이다. 94장면·403개 CoC에서 행동 방향이 분명한 134개 사건을 고른다. 130건은 이후 속도 변화를 찾아야 하고, 4건은 사건 시점에 이미 정지·양보를 만족한다. 제한 시간, 반복 사건에서 시간을 재기 시작하는 시각, 속도 관찰 구간과 최소 변화량을 바꾸어 판정이 얼마나 달라지는지 27개 설정에서 비교한다. 마지막 다섯 후보는 추가 증거와 처리 방식 검토가 필요한 항목이다.

마지막 장면 경계 점검은 한 장면 안의 반응 검사를 두 장면 사이까지 넓힌다. 첫 장면에 아직 끝나지 않은 요구를 두고, 두 장면이 이어진다는 기록 없이 둘째 장면에 반응을 넣은 시험 시간선을 사용한다. 올바른 모델은 경계에서 이전 요구를 폐기하고, 오류를 넣은 모델은 그 요구를 둘째 장면의 반응으로 잘못 완료한다. 이 비교로 자료 연결 근거가 없을 때 이전 장면의 요구를 넘기지 않는지 확인한다.

7 실험 결과

표 11와 표 12는 각 수치가 어떤 결론을 뒷받침하고 어떤 결론까지는 뒷받침하지 않는지를 한 줄씩 보여 준다. 먼저 합성 비교로 중심 주장을 확인하고 Python과 UPPAAL의 결과가 같은지 살핀다. 이어 실제 주행 자료에서 여러 사람이 주석을

표 12. 보조 결과와 직접 지지되는 주장.
Table 12. Auxiliary results and directly supported claims.

Result	Supported claim	Not supported
scene-0001: reference case passes; four modified cases fail $134 = 125 + 4 + 5$ at $D = 3$ s; two decisions reverse 130 targets/27 settings: 72 unchanged, 58 varied Correct boundary handling passes; faulty carryover fails	Checker distinguishes four deliberately added errors Deadline and repeated-event rules change results Speed-based evidence depends on its settings Unsupported transfer to the next scene is blocked	Full-dataset performance or natural error rate Safety or the best repeat rule Physical validity of every setting Long-horizon or object- continuity accuracy

얼마나 비슷하게 해석하는지 확인하고, 반응 시간 설정과 장면 경계 처리에 따라 결과가 달라지는 범위를 설명한다.

7.1 이전 요구 기억 여부를 비교한 합성 시험

표 13에서 이전 요구를 기억하는 검사는 바꾼 사례 40건 모두에서 미리 지정한 오류를 판정했고, 현재 사건만 보는 검사는 20건에서만 오류를 판정했다. 두 검사 모두 정상 기준 사례 40건에는 오류를 보고하지 않았다. 차이는 이전 사건 정보가 꼭 필요한 두 경우에서만 나타났다. 하나는 정지·양보 요구가 끝나기 전에 진행한 경우이고, 다른 하나는 이미 끝난 요구를 검사 목록에서 지우지 않은 경우이다. 이 두 종류는 현재 사건만 보면 0/10, 이전 요구를 기억하면 10/10이었고, 현재 사건 정보만으로 판단할 수 있는 나머지 두 종류는 결과가 같았다.

표 13. 합성 기준-변형 쌍에서 이전 요구 기억 여부에 따른 판정 결과.
Table 13. Results for synthetic reference-modified pairs with and without prior-requirement memory.

Consistency error	Current only	Prior retained
Proceed while stop/yield remains	0/10	10/10
End a requirement too early	10/10	10/10
Violate the required action order	10/10	10/10
Keep a requirement after its end	0/10	10/10
All modified cases	20/40	40/40
All reference cases	0/40	0/40

이 동일 입력 비교는 정지·양보 요구가 끝나기 전 진행한 오류와 끝난 요구를 목록에 남긴 오류를 찾으려면 이전 요구를 기억해야 함을 직접 보여 준다. 바꾼 사례를 만들 때 기대한 오류가 실제로 나오는지 미리 확인했으므로, 40/40은 정해 둔 네 규칙이 시험 사례에서 모두 실행됐다는 뜻이다. 처음 보는 자연 오류를 얼마나 정확히 찾는지는 별도로 평가해야 한다.

7.2 UPPAAL 구현 일치

다음으로 같은 검사 규칙을 Python과 UPPAAL에서 실행했을 때 결과가 같은지 확인했다. 실제 UPPAAL 검사 프로그램을 실행한 일곱 표준 시험 사례에서 최종 판정, 오류 종류, 첫 위반 사건과 미상 사유가 모두 일치하여 불일치는 0/7이었다. 정지 요구가 끝나기 전에 진행한 사례에서는 끝나지 않은 정지 요구가 목록에 남은 상태에서 진행 사건을 읽자, 그 사건을 첫 위반으로 기록했다. 따라서 두 구현은 시험한 사례에서 같은 판정과 위반 기록을 재현했다. 다만 둘 다 같은 규칙을 구현하므로, 이 결과가 규칙 자체의 옳음이나 가능한 모든 주행 환경의 안전을 증명하지는 않는다.

7.3 자연 표본 검토

표 14. 27개 선택 장면에 대한 합의 전 검토자 일치도.
Table 14. Reviewer agreement before discussion on 27 selected scenes.

Review item	Pair agreement	Fleiss κ
Temporal requirement	0.698	0.203
Required action sequence	0.340	0.291
Textual consistency	0.407	0.027

표 14처럼 독립 판정에서 텍스트 일관성의 검토자 쌍 일치 비율은 0.407이었다. Fleiss κ 는 우연히 같은 답을 고를 가능성을 뺀 일치 정도로, 1에 가까울수록 의견이 잘 맞고 0에 가까울수록 거의 맞지 않는다. 이 값은 0.027이었으며, 텍스트 일관성 항목은 23/27장면에서 의견이 같았다. 세 검토 항목 중 하나 이상에서는 27/27장면 모두 의견 차이가 있었다. 근거를 함께 검토한 뒤에는 텍스트 일관 27/27, 모순 0/27로 합의했다. 이는 자동 주석을 사람마다 다르게 읽을 수 있어 합의 절차가 필요함을 보여 주며, 0/27은 선택한 장면에 대한 검토자 합의 기록이다.

표 15. 합의된 문장 판정과 기록 궤적 판정의 장면별 교차표.
Table 15. Scene-level comparison of agreed text decisions and recorded-trajectory decisions.

Consensus text	Aligned	Not aligned	Unknown
Consistent	2	7	18
Contradictory	0	0	0

표 15에서 문장들은 모두 서로 모순이 없다고 합의됐지만, 문장이 요구한 행동과 기록된 이동 경로의 관계는 2장면 일치, 7장면 불일치, 18장면 판단에 필요한 정보 부족(미상)이었다. 이는 문장끼리 모순이 없는지와 문장이 기록된 행동에 맞는지가 서로 다른 문제임을 보여 준다. 불일치 7건도 상대 객체가 요구 종료 조건을 만족했는지와 차량의 실제 움직임을 설명할 정보가 없으므로, 교통법규나 물리적 안전을 위반했다고 판단할 수 없다.

7.4 보조 검사기의 의도적 변경 시험과 반응 설정 결과

앞선 결과가 문장 사이의 모순에 답했다면, 다음 결과는 반응 시간과 자료 출처 처리 방식이 검토 결과를 어떻게 바꾸는지 보여 준다. **scene-0001**의 첫 감속 요구는 2.5초, 차차 속도에서 찾은 반응 시작은 4.6597초여서 약 2.1597초가 걸렸다. 표 16의 변경하지 않은 기준 모델은 제한 시간 $D = 3$ 초에서 모든 검사 조건을 통과했다. 제한 시간을 2초로 줄이거나, 반복 사건에서 시계를 다시 시작하거나, 반응을 제거하거나, 낮은 품질 사건이 기존 정보를 덮어쓰게 했을 때는 각각 관련 검사 조건이 실패했다. 표에서 P는 통과(pass), F는 실패(fail)를 뜻하며, 여섯 열은 요구 상태, 반응 확인, 제한 시간, 반복 시계, 정보 덮어쓰기와 멈춤 없는 실행을 차례로 나타낸다.

표 16. **scene-0001**의 기준 모델과 한 항목씩 바꾼 모델의 검사 결과.
Table 16. Check results for the reference scene-0001 model and four one-item modifications.

Case	Req.	Resp.	Time	Repeat	Source	Run
reference	P	P	P	P	P	P
2 s limit	P	P	F	P	P	P
restart time	P	P	P	F	P	P
no response	P	F	F	P	P	P
rejected overwrite	P	P	P	P	F	P

바꾼 내용과 관련된 검사 조건만 실패했으므로, 기준 모델이 단순히 검사를 꺼 두어서 통과한 것은 아니다. 즉 검사기는 의도적으로 넣은 네 오류를 구분했다. 이어 전체 방향성 사건에서 설정을 바꿀 때 판정이 얼마나 달라지는지 확인했다.

반응 시간 보조 분석에서 행동 방향이 분명한 134건은 이후 속도 변화를 찾아야 하는 130건과, 사건 시점에 이미 정지·양보 요구를

만족한 4건으로 구성된다($134 = 130 + 4$). 기본 제한 시간 3초와 첫 판단 시각부터 계속 시간을 재는 방식에서 130건 중 125건은 속도 반응을 찾았고 5건은 검토 후보였다. 이미 만족한 4건을 더하면 $134 = 125 + 4 + 5$ 이다. 처음에는 후보가 8건이었지만, 사건 시점 속도가 $v \leq 0.5$ m/s인 정지·양보 요구는 이미 충족한 것으로 보는 탐색 규칙을 나중에 적용해 세 건을 다시 분류했다. 따라서 129건은 이 검사 설정을 통과한 항목이고, 다섯 항목은 추가 검토가 필요하다.

표 17. 반복 사건의 시간을 재는 두 방식에서 다섯 검토 후보가 통과한 제한 시간.
Table 17. Passing deadlines for five review candidates under two ways of timing repeated events.

Scene	Required action	Passing D (s): first-event/restart-on-repeat	Observation
0019	Yield/decelerate	none / none	Response found in 4/27 settings
0028	Red-light stop	4,5 / 3,4,5	Reversal at $D = 3$ s
0042	Lead-car deceleration	4,5 / 4,5	Changes at 3–4 s
0065	Bus deceleration	4,5 / 4,5	Other-object state unavailable
0074	Route acceleration	4,5 / 3,4,5	Reversal at $D = 3$ s

표 17에서 **scene-0028**의 반응은 첫 감속 판단 약 3.24초 뒤, 반복 판단 1.24초 뒤에 나타났다. **scene-0074**의 반응은 첫 가속 판단 약 3.36초 뒤, 반복 판단 0.86초 뒤에 나타났다. 제한 시간이 3초일 때 첫 판단 시각을 유지하면 두 장면 모두 실패하지만, 반복 판단에서 시계를 다시 시작하면 통과한다. 따라서 반복 사건에서 어느 시각부터 시간을 켤지 미리 정해야 한다. **scene-0042**와 **scene-0065**도 제한 시간 3초에서는 실패하고 4초부터 통과하여 제한 시간 선택의 영향을 보여 준다.

표 18. 130개 사건에서 27개 속도 변화 판정 설정이 반응을 찾은 횟수.
Table 18. Number of response settings that find a speed change for each of 130 events.

Settings finding a response	Event count
27/27	72
18–26/27	51
9–17/27	2
1–8/27	4
0/27	1

표 18에서 72건은 27개 설정 모두가 반응을 찾았지만, 58건은 설정에 따라 판정이 달랐다. 한 후보는 어떤 설정에서도 반응을 찾지 못했고, 네 후보는 1–8개 설정에서만 반응을 찾았다. 즉 자차 속도에서 반응을 찾은 결과는 살펴보는 시간 구간, 최소 속도 변화와 요구 방향 일치 기준에 따라 달라진다. 따라서 검토 결과에는 사용한 탐지 설정도 함께 기록해야 한다.

표 19. 연결 근거 없이 이전 장면의 요구를 넘기는 오류를 막는 시험.
Table 19. Test for blocking prior-requirement carryover without scene-continuity evidence.

Case	Finished	Reset	No cross-scene	No carryover	Run
correct	P	P	P	P	P
faulty transfer	P	F	F	F	P

마지막으로 표 19에서도 P와 F는 각각 통과와 실패를 뜻한다. 올바른 모델은 두 장면이 이어진다는 기록이 없는 경계에서 이전 요구를 폐기하고 모든 검사 조건을 통과했다. 오류를 넣은 모델은 첫 장면의 요구를 둘째 장면의 반응으로 잘못 완료했기 때문에, 경계에서 폐기해야 한다는 조건과 장면 사이 완료·이월을 금지한 조건을 위반했다. 따라서 자료 연결 근거가 없는 장면 결합을 막는 규칙이 의도대로 작동함을 확인했다.

8 논의

합성 시험에서 현재 사건만 보는 검사가 놓친 두 오류는 이전 요구를 기억해야만 찾을 수 있었다. 첫 번째는 “계속 정지하라”는 요구가 아직 끝나지 않았는데 주행을 시작한 문장 사이의 충돌이다. 두 번째는 정지 요구가 끝났는데도 이를 검사 목록에서 지우지 않아 이후의 정상적인 진행을 오류로 판단하는 검사 처리 문제이다. 반면 요구를 너무 일찍 끝낸 오류와 행동 순서를 어긴 오류는 한 사건 안에 필요한 정보가 모두 있어 두 검사 모두 찾았다. 따라서 40/40 대 20/40의 차이는 여러 사건에 걸쳐 요구가 이어지는 CoC에서는 이전 요구를 기억하는 검사가 분명한 효과가 있음을 뒷받침한다.

이 효과를 실제 자료 관리에 활용하기 위해, 검사 결과에 따라 학습 자료의 검토 순서를 정하는 절차를 제안한다. “검사 통과”는 미리 정한 일관성 오류를 찾지 못했다는 뜻이고, “오류 발견” 또는 “판단할 정보가 부족함(미상)”은 사람이 다시 보거나 증거를 보충해야 한다는 뜻이다. 판정 이유와 결과를 바꾼 조건을 함께 남기면 자료를 자동으로 버리지 않고도 먼저 살펴볼 항목을 정할 수 있다.

표 20. 검사 근거에 따른 학습 전 자료 분류와 후속 조치.
Table 20. Proposed pre-training grouping and follow-up action based on checking evidence.

Evidence	Proposed action
Source and checking evidence preserved; no consistency error found under the stated rules	Keep the item and its decision record
Decision changes with a rule or response setting	Review the item and record the setting that changed it
Unknown release/object evidence	Request additional annotation or sensor evidence
Missing scene linkage or continuity	Do not join the scenes without source evidence
Known modified synthetic case	Use only to test the checker

표 20의 분류는 자료를 자동으로 채택하거나 삭제하기 위한 것이 아니다. 판정 근거를 보존하면서 사람이 검토할 순서를 정하기 위한 것이다. 예를 들어 반복 CoC의 시간을 첫 판단 시각부터 계속 켤 때와 반복 판단 시각부터 다시 켤 때 결과가 다르면, 사용한 시간 기준과 바뀐 결과를 함께 저장한다. 요구 종류와 관련된 객체 정보가 미상이면 임의로 정상이라고 채우지 않는다. 두 장면이 실제로 이어진다는 근거가 없으면 앞 장면의 요구를 다음 장면으로 넘기지 않는다. 의도적으로 바꾼 합성 사례는 학습 자료에 섞지 않고 검사기가 정해 둔 규칙대로 작동하는지 확인하는 데만 쓴다. 이 분류 절차가 실제 학습 성능을 높이는지는 본 논문에서 시험하지 않았으므로 향후 검증이 필요하다.

보조 결과는 왜 판정 조건을 함께 기록해야 하는지도 보여 준다. 제한 시간을 $D = 3$ 초로 두었을 때 134건은 속도 변화 확인 125건, 이미 요구를 만족한 4건, 추가 검토가 필요한 5건으로 나뉘었다. 이 가운데 두 장면은 첫 판단 시각부터 시간을 재면 실패했지만 반복 판단 시각부터 다시 재면 통과했다. 속도 변화를 찾은 27개 설정에서도 결과 차이가 나타났다. 이와 별도로인 장면 경계 시험에서는 연결 근거가 없는 장면을 합칠 때 한 장면의 반응을 다른 장면의 반응으로 잘못 볼 수 있음을 확인했다. 이 보조 결과는 중심 합성 시험과 다른 질문을 다루지만, 품질 검사 결과를 곧바로 안전 여부로 해석해서는 안 되는 이유를 분명히 한다.

실제 주행 자료를 이용한 검토 결과는 다음 연구 과제도 보여 준다. 검토자 사이의 텍스트 일치도 $\kappa = 0.027$ 은 같은 지침을 사용해도 처음부터 같은 판단을 내리기 어려웠음을 뜻한다. 서로 다른 판단의 이유를 함께 검토한 뒤에는 선택한 27개 장면 모두에서 모순이 없다는 데 합의했다. 후속 연구에서는 자연스럽게 발생하는

모순·비모순 자료로 시험 범위를 넓히고, 객체 상태와 이동 경로 증거 및 학습 전후 성능을 함께 측정하여 실제 오류를 얼마나 잘 찾고 학습에 얼마나 도움이 되는지 확인해야 한다.

9 한계와 결과 해석 시 주의점

9.1 자료의 한계

CoC는 사람이 직접 작성한 안전 규칙이 아니다. 주행이 끝난 뒤 기록 영상과 이후 이동 경로를 바탕으로 자동 생성한 설명이다. 따라서 장면의 실제 원인이나 운전 의도를 정확히 설명한다는 보장은 없다. 현재 보관된 변환 자료에는 설명을 만든 모델과 자동 표시 도구의 정확한 버전도 일부 남아 있지 않다. 또한 선택한 27개 장면은 무작위 표본이 아니며 영상과 객체 상태도 제공하지 않으므로, 본 연구는 문장 사이의 관계만 평가했다.

9.2 평가 범위의 한계

합성 40쌍은 실제 오류를 따로 수집한 평가 자료가 아니다. 정상 사례에서 한 항목을 의도적으로 바꾸고, 예상한 판정이 나오도록 미리 확인한 시험 사례이다. 따라서 이전 요구를 기억하는 검사의 40/40과 현재 사건만 보는 검사의 20/40은 정해 둔 네 오류 규칙을 얼마나 빠짐없이 실행했는지를 보여 줄 뿐이다. 실제 오류를 맞힌 비율, 실제 오류를 놓치지 않은 비율, 정상 자료를 오류로 잘못 판단한 비율은 아니다. 네 검토자의 첫 판단도 서로 잘 맞지 않았고, 제3자가 모든 합의 과정을 그대로 재현할 기록은 부족하다. 그러므로 검토 후 27/27이 일치했다는 결과도 객관적인 정답으로 볼 수는 없다.

보조 반응 검사는 실제 브레이크나 조향 명령을 사용하지 않고 자차 속도 변화로 반응을 대신 판단했다. 관찰 구간 W , 필요한 속도 변화량 δ , 변화 방향의 일관성을 나타내는 비율 ρ 에 따라 결과가 달라지며, 시험한 27개 조합도 가능한 모든 설정을 포함하지 않는다. 제한 시간 $D = 3$ 초와 STOP/YIELD에서 $v \leq 0.5$ m/s이면 이미 요구를 만족했다고 보는 규칙도 별도의 차량 안전 기준에서 가져온 것이 아니다. 특히 후자는 초기 후보를 살펴본 뒤 추가한 탐색 규칙이다. 따라서 $134 = 125 + 4 + 5$ 는 명시한 설정에서 자료가 어떻게 나뉘었는지를 나타낼 뿐, 차량의 안전률이나 오류율이 아니다.

9.3 구현과 자료 출처의 한계

문장에서 정보를 읽는 부분, 입력 자료를 점검하는 부분, 서로 다른 행동 표현을 하나로 맞추는 부분, 결과를 분류하는 부분의 구현 오류가 판정에 영향을 줄 수 있다. scene-0001을 의도적으로 바꾼 시험은 검사기가 미리 넣은 네 오류를 구분하는지만 확인하며, 아직 알려지지 않은 오류까지 찾는 능력은 측정하지 않는다. 현재 자료에는 장면 사이의 기록, 앞뒤 관계, 동일 객체가 이어지는지 보여 주는 정보가 없다. 그래서 기본적으로 각 장면을 서로 독립된 주행 구간으로 처리했다. 장면 경계 시험은 이 처리 방식이 코드에 올바르게 구현됐는지만 확인하며, 실제로 장면 사이에 연속 주행이 없었다는 증거는 아니다.

9.4 UPPAAL 모델과 일반화의 한계

UPPAAL 모델은 미리 정한 사건 순서를 그대로 재생하고 Python 검사기와 같은 상태 변경 규칙을 사용한다. 7개 시험 사례에서 결과가 모두 같았다는 사실은 두 구현이 해당 사례에서 일치했음을 보여 주지만, 규칙 자체가 현실을 올바르게 표현한다는 독립적인 증명은 아니다. 상대 객체의 상태, 실제 시간 제한, 브레이크·조향 명령, 차량의 물리적 움직임과 제어 결과가 다시 차량에 영향을 주는 과정도 포함하지 않았다. 따라서 다른

자료·언어·CoC 생성 방식으로의 일반화, 실제 차량 안전, 제어 명령 생성 또는 현장 적용 가능성을 입증하지 않는다.

10 결론

본 논문은 연속 CoC에서 이전 사건의 요구, 요구가 끝나는 조건, 행동 순서, 그리고 참·거짓·정보 부족으로 나뉘는 증거를 읽어 검사 규칙으로 만들었다. 이 규칙은 이전 요구를 기억하는 Python 검사기와 같은 방식으로 동작하는 UPPAAL 검사 모델에서 실행했다. 중심 결론은 분명하다. 본 연구에서 정한 규칙과 합성 시험 사례에서는 이전 요구를 기억해야 “정지 요구가 끝나기 전에 진행한 경우”와 “끝난 요구를 검사 목록에서 지우지 않은 경우”를 모두 구분할 수 있다.

가장 직접적인 근거는 정상 사례와 한 항목을 바꾼 사례로 이루어진 합성 시험 40쌍이다. 이전 요구를 기억하는 검사는 바꾼 사례 40/40에서 미리 지정한 오류를 판정했고, 현재 사건만 보는 검사는 20/40만 찾았다. 정상 사례를 오류로 잘못 판정한 경우는 두 검사 모두 0/40이었다. 두 검사의 차이인 20건은 모두 앞 문장의 요구를 알아야 판단할 수 있는 사례였다. 즉, 요구가 끝나기 전에 진행했거나 끝난 요구를 검사 목록에서 지우지 않은 경우였다. 또한 Python과 UPPAAL의 7개 표준 시험 사례에서는 최종 판정, 오류 종류, 최초 위반 사건과 정보 부족 사유가 모두 같아 불일치가 0/7이었다. 이는 두 구현이 시험한 범위에서 같은 판정과 위반 기록을 냈음을 보여 준다.

실제 주행 자료에서는 90개 장면에서 이어지는 사건 두 개씩을 묶은 309개 구간을 만들고, 그중 27개 장면을 검토했다. 검토자들의 첫 판단이 얼마나 비슷한지를 나타내는 Fleiss κ 는 0.027로 낮았고, 서로 다른 판단의 이유를 논의한 뒤에는 27개 모두에서 모순이 없다는 데 합의했다. 이는 실제 오류의 비율을 계산한 결과가 아니라, 자동 생성 설명이 모호할 수 있으며 사람 사이의 합의 과정이 필요함을 보여 주는 결과이다.

보조 분석에서는 scene-0001에서 한 항목씩 바꾼 네 사례가 모두 예상한 검사 조건에서 실패했다. 행동 방향이 분명한 134개 사건은 기본 설정에서 속도 변화 확인 125건, 사건 시점에 이미 요구를 만족한 4건, 추가 검토가 필요한 5건으로 나뉘었다. 제한 시간을 $D = 3$ 초로 두었을 때 두 장면은 첫 판단 시각부터 시간을 재면 실패하고 반복 판단 시각부터 다시 재면 통과했다. 속도로 반응을 판단한 130건 중 58건은 27개 속도 판정 설정에서 같은 결과를 유지하지 않았다. 이와 별도의 장면 경계 시험은 연결 근거 없이 앞 장면의 요구를 다음 장면으로 넘기는 오류를 구분했다. 이 결과는 검사 조건, 속도에서 계산한 증거, 장면 연결 근거를 함께 관리해야 함을 보여 주지만, 실제 자료의 오류를 얼마나 잘 찾는지를 직접 측정한 것은 아니다.

종합하면 본 결과는 여러 사건에 걸쳐 요구가 이어질 때 이전 요구를 기억하는 검사가 필요하고 효과적임을 뒷받침한다. 또한 같은 검사 규칙을 Python과 UPPAAL로 실행할 수 있으며, 시험 사례에서는 두 구현의 최종 판정과 위반 기록이 같음을 확인했다. 다음 단계는 별도로 수집한 실제 오류 자료에 객체 상태와 이동 경로 증거를 더하고 학습 전후 성능을 비교하여, 이 효과가 실제 오류 검출과 더 정확한 행동 학습으로 이어지는지를 평가하는 것이다.

감사의 글

본 연구는 대한민국 정부(교육과학기술부)의 재원으로 한국연구재단(NRF)의 지원을 받아 수행되었으며(NRF-2023R1A2C1006639), 대한민국 정부(과학기술정보통신부)의 재원으로

참고문헌

- [1] Yihan Hu, Jiazhi Yang, Li Chen, Keyu Li, Chonghao Sima, Xizhou Zhu, Siqi Chai, Senyao Du, Tianwei Lin, Wenhai Wang, Lewei Lu, Xiaosong Jia, Qiang Liu, Jifeng Dai, Yu Qiao, and Hongyang Li. Planning-Oriented Autonomous Driving. In Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), pages 17853–17862, 2023.
- [2] Bo Jiang, Shaoyu Chen, Qing Xu, Bencheng Liao, Jiajie Chen, Helong Zhou, Qian Zhang, Wenyu Liu, Chang Huang, and Xinggang Wang. VAD: Vectorized scene representation for efficient autonomous driving. In Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), pages 8340–8350, 2023.
- [3] Chonghao Sima, Katrin Renz, Kashyap Chitta, Li Chen, Hanxue Zhang, Chengen Xie, Jens Beisswenger, Ping Luo, Andreas Geiger, and Hongyang Li. DriveLM: Driving with graph visual question answering. arXiv preprint arXiv:2312.14150, 2023.
- [4] Xiaoyu Tian, Junru Gu, Bailin Li, Yicheng Liu, Yang Wang, Zhiyong Zhao, Kun Zhan, Peng Jia, Xianpeng Lang, and Hang Zhao. DriveVLM: The convergence of autonomous driving and large vision-language models. In Proc. 8th Conf. Robot Learning (CoRL), volume 270 of Proceedings of Machine Learning Research, pages 4698–4726. PMLR, 2025.
- [5] Yan Wang, Wenjie Luo, Junjie Bai, Yulong Cao, Tong Che, Ke Chen, Yuxiao Chen, Jenna Diamond, Yifan Ding, Wenhao Ding, et al. Alpamayo-R1: Bridging reasoning and action prediction for generalizable autonomous driving in the long tail. arXiv preprint arXiv:2511.00088, 2025.
- [6] Nicanor Mayumu, Xiaoheng Deng, and Patrick Mukala. Is VLA reasoning faithful? probing safety of chain-of-causation in autonomous driving models. arXiv preprint arXiv:2605.17268, 2026.
- [7] Thach Nguyen, Danhua Guo, Tom Lampo, Fei Wu, and Burhan Yaman. VLADriveBench: Evaluating CoT-action relationship in VLA for autonomous driving. arXiv preprint arXiv:2606.12706, 2026.
- [8] Abhinaw Priyadershi and Jelena Frtunikj. Lost in fog: Sensor perturbations expose reasoning fragility in driving VLAs. arXiv preprint arXiv:2605.21446, 2026.
- [9] Kalpana Panda, Wesley Maia, Vinti Agarwal, and Ross Greer. What do they see? interpreting complex road scenarios through the eyes of vision-language-action models for safe and trustworthy autonomous vehicle learning. arXiv preprint arXiv:2607.16938, 2026.
- [10] Gerd Behrmann, Alexandre David, and Kim G. Larsen. A tutorial on UPPAAL. In Formal Methods for the Design of Real-Time Systems, volume 3185 of Lecture Notes in Computer Science, pages 200–236. Springer, 2004.
- [11] Daniel J. Fremont, Tommaso Dreossi, Shromona Ghosh, Xiangyu Yue, Alberto L. Sangiovanni-Vincentelli, and Sanjit A. Seshia. Scenic: A language for scenario specification and scene generation. In Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI), pages 63–78, 2019.
- [12] Tommaso Dreossi, Daniel J. Fremont, Shromona Ghosh, Edward Kim, Hadi Ravanbakhsh, Marcell Vazquez-Chanlatte, and Sanjit A. Seshia. VerifAI: A toolkit for the formal design and analysis of artificial intelligence-based systems. In Proc. Int. Conf. Computer Aided Verification (CAV), pages 432–442, 2019.
- [13] Holger Caesar, Varun Bankiti, Alex H. Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. nuScenes: A multimodal dataset for autonomous driving. In Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), pages 11621–11631, 2020.
- [14] NVIDIA Corporation. Chain-of-Causation (CoC) Autolabeling Pipeline. GitHub repository, 2026. Accessed Aug. 1, 2026.